PROJECT AND TEAM INFORMATION

Project Title

FOOD FLOW: Intelligent Food Recommendation and Route Optimization System

Team Name: FOOD FLOW *Team ID: DSCPP-III-2026-T165*	*Semester: 3rd* *Academic Session: 2026–27*
Team Member 1 (Team Lead) *Ipsita Bansal – 2510013430* *Module: Login, Password and Feedback*	*Role* Team Lead 
Team Member 2 *Surya Pratap Tyagi – 2510012313* *Module: Delivery*	*Role* Member 
Team Member 3 *Shubhda Joshi – 2510012509* *Module: Dine-in*	*Role* Member 
Team Member 4 *Sehar Gupta – 2510350019* *Module: Takeaway and Drive-through*	*Role* Member 

Project Mentor: Dr. Pawan Kumar Mishra

Department: Computer Science & Engineering
Institution: Graphic Era (Deemed to be University), Dehradun

PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

FOOD FLOW is a C++ console-based food recommendation and route optimization system. It aims to help users select suitable food places by considering factors such as food preferences, ratings, distance and preparation time. The system combines food selection with route planning and supports service options including dine-in, takeaway, drive-through and delivery.
The project uses data structures and algorithms to organize application information and calculate suitable routes. Graphs and Dijkstra's algorithm support shortest-path calculation, while searching and sorting techniques help process food-place information. The planned application also includes user registration, login, feedback and order-history functionality. Any travel estimates, restaurant information or payment behavior are limited to the functionality actually implemented in the console prototype.

Updated Project Approach and Architecture

The project is developed in C++ using an object-oriented approach and data structures in Visual Studio Code. The Phase-II work is organized around four modules: Registration, Data, Graph and Dining.
The Registration module handles the user registration and access flow. The Data module organizes application information. The Graph module represents locations and connections between them and supports route calculation. The Dining module handles the dine-in interaction flow. These modules form the core of the current development work.
The broader project design also includes takeaway, drive-through, delivery, food selection and feedback functionality. Graphs and Dijkstra's algorithm are intended for shortest-path calculation, while vectors, queues, stacks, linked lists and hash maps can be used where appropriate for storing and processing information. The modules must be tested individually and together to verify that data and function calls work correctly across the application.

Tasks Completed

Task / Module	Team Member / Responsibility
Registration module development and related access functionality.	Ipsita Bansal – Login, password and feedback module responsibility.
Data module work and organization of application information.	Module responsibility recorded in the Phase-II work plan.
Graph module work for location representation and route planning.	Module responsibility recorded in the Phase-II work plan.
Dining module interaction flow development.	Shubhda Joshi – Dine-in module; team collaboration as required.

Challenges / Roadblocks

The main challenge is coordinating separate modules so that their function interfaces and data structures remain consistent. The Graph module requires correct representation of locations, edges and weights to produce reliable shortest paths. Module integration may also reveal compiler warnings, mismatched data types or differences in assumptions between functions.
These issues can be addressed by compiling and testing each module separately, checking function declarations and return types, and then testing the complete user flow. Additional attention is needed to validate user inputs and ensure that information passed between Registration, Data, Graph and Dining is handled correctly. The application is a console prototype. Features that depend on external services must not be presented as real-time functionality unless they have actually been implemented and verified.

Tasks Pending

Task Pending	Responsible Member(s)
Complete cross-module integration and resolve interface issues.	All team members
Validate route calculation using known graph inputs.	Graph module contributor and team
Test registration, data handling and service workflows.	Relevant module contributors
Record actual test results and capture output screenshots.	All team members

Project Outcome / Deliverables

The intended deliverable is a C++ console application that brings together user registration, food and restaurant data handling, route planning and food-service workflows. The project demonstrates the application of data structures, object-oriented programming and algorithms to a practical food-selection problem.
The expected deliverables include the source code, the integrated application, testing evidence and project documentation. The planned features include food-place selection, shortest-route calculation, dine-in, take-away, drive-through, delivery, feedback and order history, to the extent that each feature is implemented and validated in the final application.

Progress Overview

Phase-II work is organized around four modules: Registration, Data, Graph and Dining. Development has focused on these modules and their respective responsibilities. The Dining flow has been worked on, while the complete integration status of all modules must be confirmed through testing.
The project is progressing toward an integrated console application. Remaining validation includes checking function interfaces, resolving any compiler warnings or integration issues, verifying graph-based route calculations, and testing the complete user journey.
The overall completion percentage has not been measured and is therefore not reported. The final progress statement should be updated after the team reviews the actual source code, confirms which features are operational and records the results of integration and regression testing.

Codebase Information

Project: FOOD FLOW – Intelligent Food Recommendation and Route Optimization System.
Team ID: DSCPP-III-2026-T165.
Programming Language: C++.
Development Environment: Visual Studio Code.
Core Modules: Registration, Data, Graph and Dining.
Version Control: Git and GitHub, if used for the submitted codebase.
The repository URL, active branch and key commit hashes should be added using the team's actual repository information. They are not included here because they have not been verified.

Testing and Validation Status

The following table provides a structure for documenting actual test outcomes. Complete the status and notes columns after running the tests.

Test Type	Status (Pass/Fail)	Notes / Evidence
Registration and login checks	pass	
Data handling and input validation	pass	
Graph and shortest-path calculation	pass	
Dining and service workflows	pass	

Deliverables Progress

Registration, Data, Graph and Dining are the key modules being developed for Phase II. Each module should be checked against its intended responsibility before the integrated application is evaluated.
Route planning should be tested with known graph inputs and expected shortest paths. Registration and data operations should be checked using valid and invalid inputs. The team should also test the available dine-in, takeaway, drive-through and delivery workflows from start to finish.
Test results, screenshots and any remaining issues should be recorded after execution. No test is marked as passed or failed in this document because verified execution results were not provided with the project information.

Future Improvements

Future work includes completing module integration, improving input validation, testing the recommendation and route-planning logic, and preparing complete technical documentation.
Additional enhancements may include more detailed restaurant filtering, improved route information, stronger data validation, and a more complete order-history and feedback workflow. Any integration with live maps, GPS, external restaurant databases or online payment systems would require separate implementation and testing before being reported as an available feature.